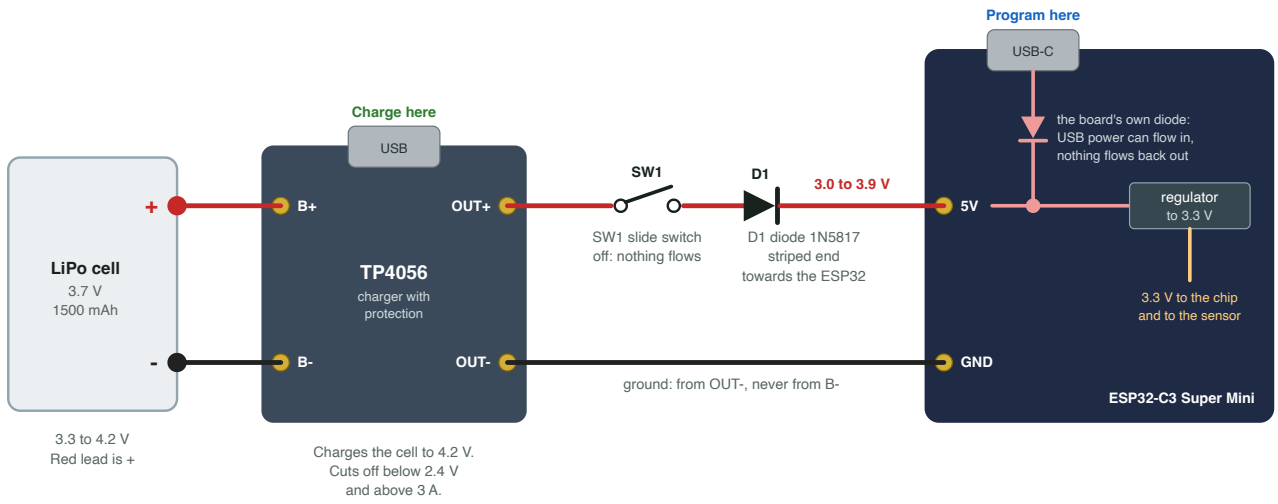
Stage 6: run from the battery

You add: the LiPo cell, the TP4056 charger module, the slide switch and the diode.

You learn: where the power flows, how the cell is charged and protected, and how to tell whether a power circuit is strong enough.

Stage 6: where the power flows

The cell is charged by the TP4056 and feeds the ESP32 through the switch and the diode.



Why the diode. The ESP32 board joins its 5V pin to its USB socket through a diode of its own. Without D1, plugging in that USB cable would push 5 volts straight into the cell, with nothing to limit the current. D1 lets current flow out of the battery only. With D1 fitted, the battery and the ESP32's USB cable can be connected at the same time. Whichever gives the higher voltage feeds the board.

Charging. Plug a phone charger into the TP4056's own socket, with the switch off. One light shows while it charges, the other when it is full.
Ground. B- is the cell's own minus. OUT- is the same point after the protection switch. Everything else connects to OUT-, so the protection can act.
Voltage. The diode costs about 0.3 V and the regulator needs about 0.1 V more than it gives. The board runs properly down to a cell voltage of about 3.5 V.

Without D1: never have the battery switched on and the ESP32's USB cable plugged in at the same time.

Read Before you touch the battery first. In this stage the order of the steps matters.

Part A: the charger and the cell, on their own

Nothing is connected to the ESP32 yet.

1. **Find out which battery lead is plus.** Set the multimeter to DC volts. Hold the red probe on the red lead and the black probe on the black lead. A reading between +3.3 and +4.2 volts means red is plus. A negative reading means the leads are the other way round.
2. **Solder the cell to the charger, one lead at a time:** plus to `B+`, minus to `B−`. Keep the bare end of the other lead taped until it is its turn.
3. **Measure from `OUT+` to `OUT−`.** It should show the cell voltage. If it shows 0, plug the charger into USB for a few seconds: its protection wakes up when it first sees power.
4. **Charge it once, fully.** Plug a phone charger into the TP4056's own socket. One light is on while it charges. The other comes on when it is full, after about two hours.

Part B: switch and diode, before the ESP32 is plugged in

5. **Take the ESP32 board out of its headers.**
6. **Solder the switch, the diode and the two leads** as in session 2 of build-on-zero-pcb.pdf. The striped end of the diode points to the `5V` pin.
7. **Measure at the empty header, from the `5V` hole to the `GND` hole:**

Switch	You should read	If not
On	0.1 to 0.4 volts less than the cell, and positive	0 volts: the diode is the wrong way round, or the switch is on the wrong pins. Negative: plus and minus are swapped. Stop and fix it.
Off	Close to 0	The switch is not in the plus wire.

8. **Only when both readings are right,** switch off and plug the ESP32 board back in.

Part C: the test

9. Plug in the USB cable and upload 06_battery_power.ino.
10. Unplug the USB cable. Switch the battery on.
11. Watch for one short flash every 2 seconds. Leave it running for at least ten minutes. Tap the board and wiggle the wires.
12. Plug the USB cable in again and open the Serial Monitor.

What you should see

```
===== Stage 6: battery power test =====
This start       power was switched on
Starts so far    1
Brownout resets  0   good, this must stay 0
Previous run     0 h 00 min 00 s
Bluetooth is advertising as "SwingLab-PowerTest", only to load the battery.
Send c to clear the counters.
running for 0 h 00 min 05 s   brownout resets 0
running for 0 h 00 min 10 s   brownout resets 0
```

The counters survive power-off, so they tell you what happened while no cable was connected. Opening the Serial Monitor can itself restart the board, which adds one to "Starts so far". That is normal.

The stage is passed when it runs from the battery with a steady heartbeat, and "Brownout resets" is still 0 afterwards.

If there has been a brownout, the board also shows it without a cable: ten fast flashes at start.

The rules from now on

- **Charge through the TP4056's socket, with the switch off.** With the bat running, the charger cannot tell when the cell is full.
- **Upload through the ESP32's socket.** With the diode fitted, the battery may stay on while you do.
- **Everything uses `OUT−` as ground, never `B−`.** Otherwise the charger's protection is bypassed.

- **Switch off when you are done.** The cell is protected against running flat, but it lasts longer if it never gets there.

What is going on

- **The cell** gives between 4.2 volts, full, and about 3.4 volts, nearly empty.
- **The charger** fills it from USB and stops by itself. Its protection chips cut the cell off if it is drained too far, overcharged or shorted.
- **The switch** opens the plus wire.
- **The diode** lets current flow towards the ESP32 only. The ESP32 board already joins its `5V` pin to its USB socket through a diode of its own. Without this second diode, plugging in the USB cable would push 5 volts into the cell with nothing to limit the current. With it, both can be connected, and the higher voltage feeds the board.
- **The regulator on the ESP32 board** turns what arrives at `5V` into 3.3 volts for the chip and the sensor.
- **A brownout** is the chip restarting because its supply dipped. The radio draws current in short, sharp bursts, and thin wires or a poor joint cannot deliver them. That is why this sketch switches Bluetooth on: to load the circuit the way the bat firmware will.
- **Preferences** is a small store in the chip's flash memory. The sketch keeps its counters there, which is how they survive power-off.

More in how-it-works.pdf.

Try this

- Leave it running from a full charge until it stops, then read "Previous run". That is close to your battery life. The bat firmware draws a little more.
- Send `c` to clear the counters before a new test.

If it does not work

See The battery.

Next: stage 7, which is optional, or straight to stage 8.